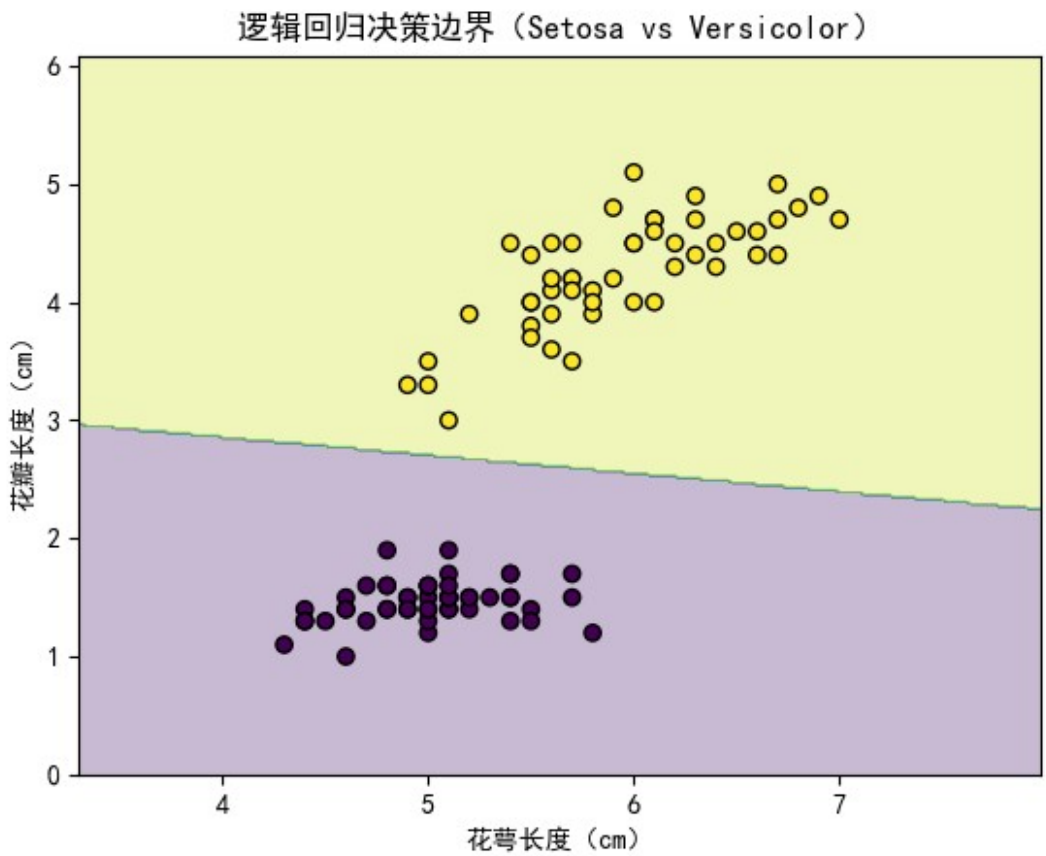
使用 Iris 数据进行逻辑回归二分类（选取前两类）

```
from matplotlib.colors import ListedColormap
from mlxtend.data import mnist_data
from sklearn.linear_model import LogisticRegression
import numpy as np

from sklearn.datasets import load_iris, fetch_openml, make_blobs
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']  # SimHei 是Windows/Linux
常见的中文字体
plt.rcParams['axes.unicode_minus'] = False  # 解决负号显示问题

# 1.加载数据并预处理（二分类）
iris = load_iris()
X = iris.data[:100, [0, 2]]
y = iris.target[:100]

# 2.划分训练集和测试集（70%训练集，30%测试集）
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 3.训练逻辑回归模型
model = LogisticRegression()
model.fit(X_train, y_train)

# 4.预测与评估
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f"测试集准确率: {accuracy:.2f}")
print("混淆矩阵：\n", conf_matrix)


# 5.可视化决策边界
def plot_decision_boundary(X, y, model):
    # 生成网格点
    h = 0.02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))

    # 预测网格点类别
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
```

```
    # 绘图
    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o')
    plt.xlabel('花萼长度（cm）')
    plt.ylabel('花瓣长度（cm）')
    plt.title('逻辑回归决策边界（Setosa vs Versicolor）')
    plt.show()


plot_decision_boundary(X, y, model)
```

```
测试集准确率: 1.00
混淆矩阵：
 [[17  0]
 [ 0 13]]
```



使用 Iris 数据进行逻辑回归多分类

```
# 1.加载完整数据集
iris = load_iris()
X = iris.data
y = iris.target
```

```
# 2.划分训练集和测试集（70%训练集，30%测试集）
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 3.训练逻辑回归模型
model = LogisticRegression(solver='lbfgs')
model.fit(X_train, y_train)

# 4.预测与评估
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f"测试集准确率: {accuracy:.2f}")
print("混淆矩阵：\n", conf_matrix)
```

```
测试集准确率: 1.00
混淆矩阵：
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

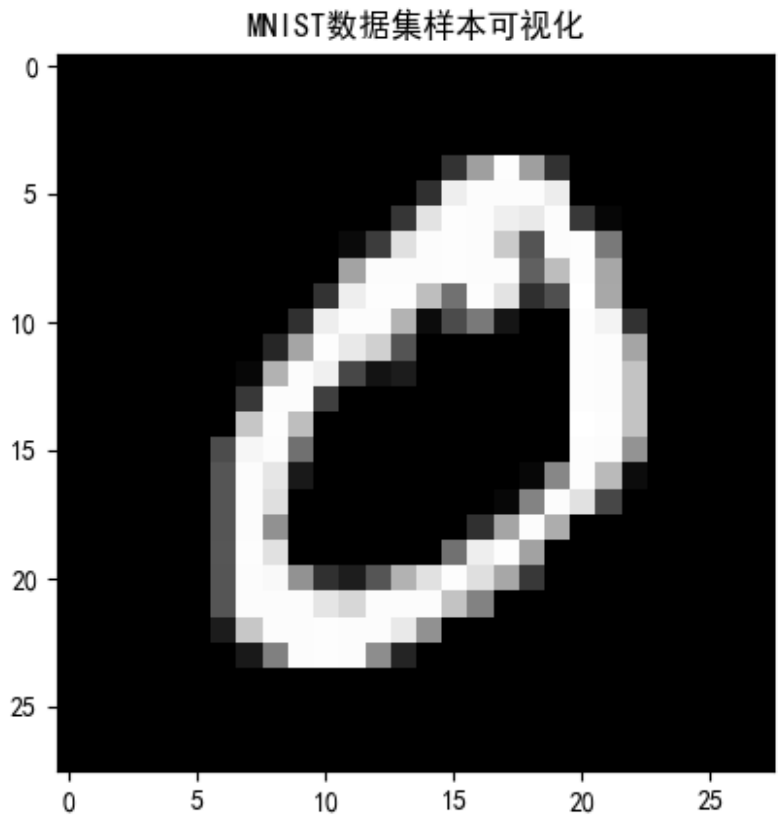
在 MNIST 数据集上应用 KNN 算法，完成分类任务。MNIST 是手写数字数据集，其中包含了很多手写数字 0~9 的黑白图像，每张图像都由 2828 个像素点组成。

```
# 加载MNIST 数据集
m_x, m_y = mnist_data()

# 数据集可视化
data = np.reshape(np.array(m_x[0], dtype=int), [28, 28])
plt.figure()
plt.title('MNIST数据集样本可视化')
plt.imshow(data, cmap='gray')

# 划分训练集和测试集（80%训练集，20%测试集）
x_train, x_test, y_train, y_test = train_test_split(m_x, m_y,
test_size=0.2, random_state=42)


class KNN:
    def __init__(self, k, label_num):
        """ 设定超参数k 和label_num """
        self.k = k
        self.label_num = label_num

    def fit(self, x_train, y_train):
        """ 在类中提前保存训练集"""
        self.x_train = x_train
        self.y_train = y_train
```

```python
    @staticmethod
    def distance(a, b):
        """ 用欧氏距离计算两个样本之间的距离"""
        return np.sqrt(np.sum(np.square(a - b)))

    def get_knn_indices(self, x):
        """ 获取离目标样本最近的k 个样本的标签"""

        # 计算已知样本的距离
        dis = list(map(lambda a: self.distance(a, x), self.x_train))

        # 按距离从小到大排序，并得到最近的k 个样本的下标
        knn_indices = np.argsort(dis)[:self.k]

        return knn_indices

    def get_label(self, x):
        """ 获取k 个近邻中数量最多的类别标签"""

        # 获取k 个近邻的下标
        knn_indices = self.get_knn_indices(x)

        # 初始化类别标签统计数组
        label_statistic = np.zeros(shape=[self.label_num])
        # 利用下标进行类别标签数量统计
        for index in knn_indices:
            label = int(self.y_train[index])
            label_statistic[label] += 1

        return np.argmax(label_statistic)

    def predict(self, x_test):
        """ 预测类别"""

        # 初始化测试集类别标签数组
        predicted_test_labels = np.zeros(shape=[len(x_test)],
dtype=int)
        # 得到测试集样本对应的类别标签
        for i, x in enumerate(x_test):
            predicted_test_labels[i] = self.get_label(x)

        return predicted_test_labels


for k in range(1, 10):
    knn = KNN(k, label_num=10)
    knn.fit(x_train, y_train)
    y_pred = knn.predict(x_test)
```

```
    accuracy = np.mean(y_pred == y_test)
    print(f"k的取值为{k}，预测准确率为：{accuracy * 100:.2f}%")
```

```
k的取值为1，预测准确率为：94.30%
k的取值为2，预测准确率为：93.70%
k的取值为3，预测准确率为：93.90%
k的取值为4，预测准确率为：93.60%
k的取值为5，预测准确率为：93.20%
k的取值为6，预测准确率为：93.20%
k的取值为7，预测准确率为：93.10%
k的取值为8，预测准确率为：92.60%
k的取值为9，预测准确率为：92.90%
```



Scikit-learn 实现 MNIST 分类

```
from sklearn.neighbors import KNeighborsClassifier

# 1. 加载MNIST 数据集
X, y = fetch_openml('mnist_784', return_X_y=True, as_frame=False)

# 2. 划分训练集和测试集（80% 训练集，20% 测试集）
X = X[:10000]
y = y[:10000]
X_train, X_test, y_train, y_test = train_test_split(X, y,
```

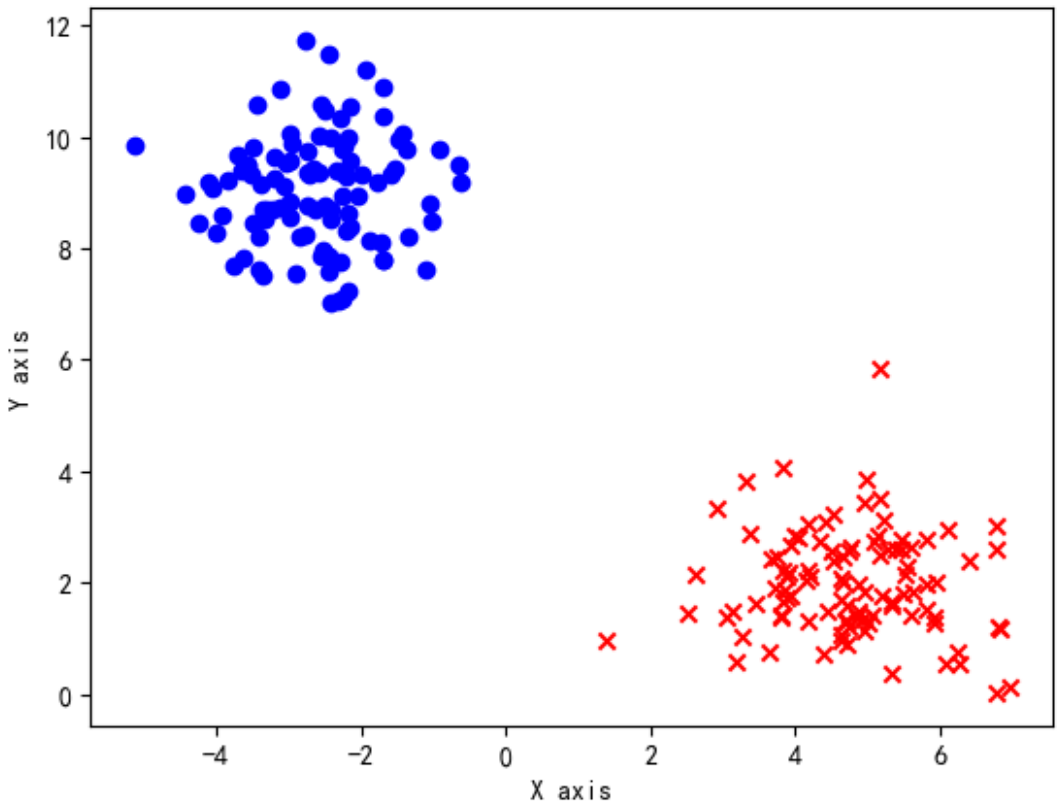
```
test_size=0.2, random_state=42)

# 3. 创建KNN 模型
knn = KNeighborsClassifier(n_neighbors=5)

# 4. 训练模型
knn.fit(X_train, y_train)

# 5. 预测与评估
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"KNN（k = 5）在 MNIST 数据集上的准确率：{accuracy * 100:.2f}%")
```

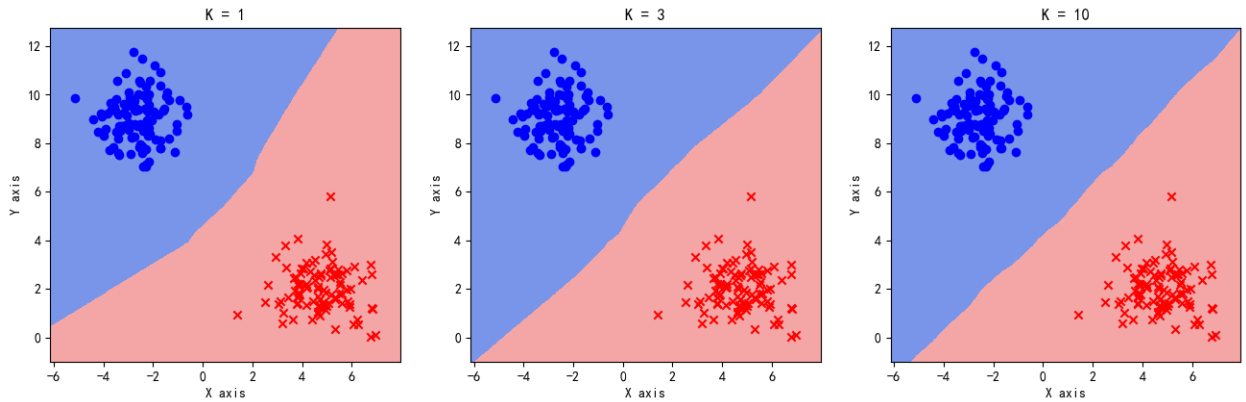
```
KNN（k = 5）在 MNIST 数据集上的准确率：94.75%
```

Scikit-learn 实现 KNN 算法

```
np.random.seed(42)
X_train, y_train = make_blobs(n_samples=200, n_features=2, centers=2,
cluster_std=1.0, random_state=42)

# 可视化
plt.figure()
# 绘制类别0 的样本（蓝色'o'）
plt.scatter(X_train[y_train == 0, 0], X_train[y_train == 0, 1],
c='blue', marker='o')
# 绘制类别1 的样本（红色'x'）
plt.scatter(X_train[y_train == 1, 0], X_train[y_train == 1, 1],
c='red', marker='x')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()

# 设置步长
step = 0.02
# 设置网格边界
x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1
# 构造网格
xx, yy = np.meshgrid(np.arange(x_min, x_max, step), np.arange(y_min,
y_max, step))
grid_data = np.concatenate([xx.reshape(-1, 1), yy.reshape(-1, 1)],
axis=1)

fig = plt.figure(figsize=(16, 4.5))
ks = [1, 3, 10]
cmap_light = ListedColormap(['royalblue', 'lightcoral'])

for i, k in enumerate(ks):
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    knn.fit(X_train, y_train)
    z = knn.predict(grid_data).reshape(xx.shape)

    ax = fig.add_subplot(1, 3, i + 1)
    ax.pcolormesh(xx, yy, z, cmap=cmap_light, alpha=0.7)
    ax.scatter(X_train[y_train == 0, 0], X_train[y_train == 0, 1],
c='blue', marker='o')
    ax.scatter(X_train[y_train == 1, 0], X_train[y_train == 1, 1],
c='red', marker='x')

    ax.set_xlabel('X axis')
    ax.set_ylabel('Y axis')
    ax.set_title(f'K = {k}')
plt.show()
```





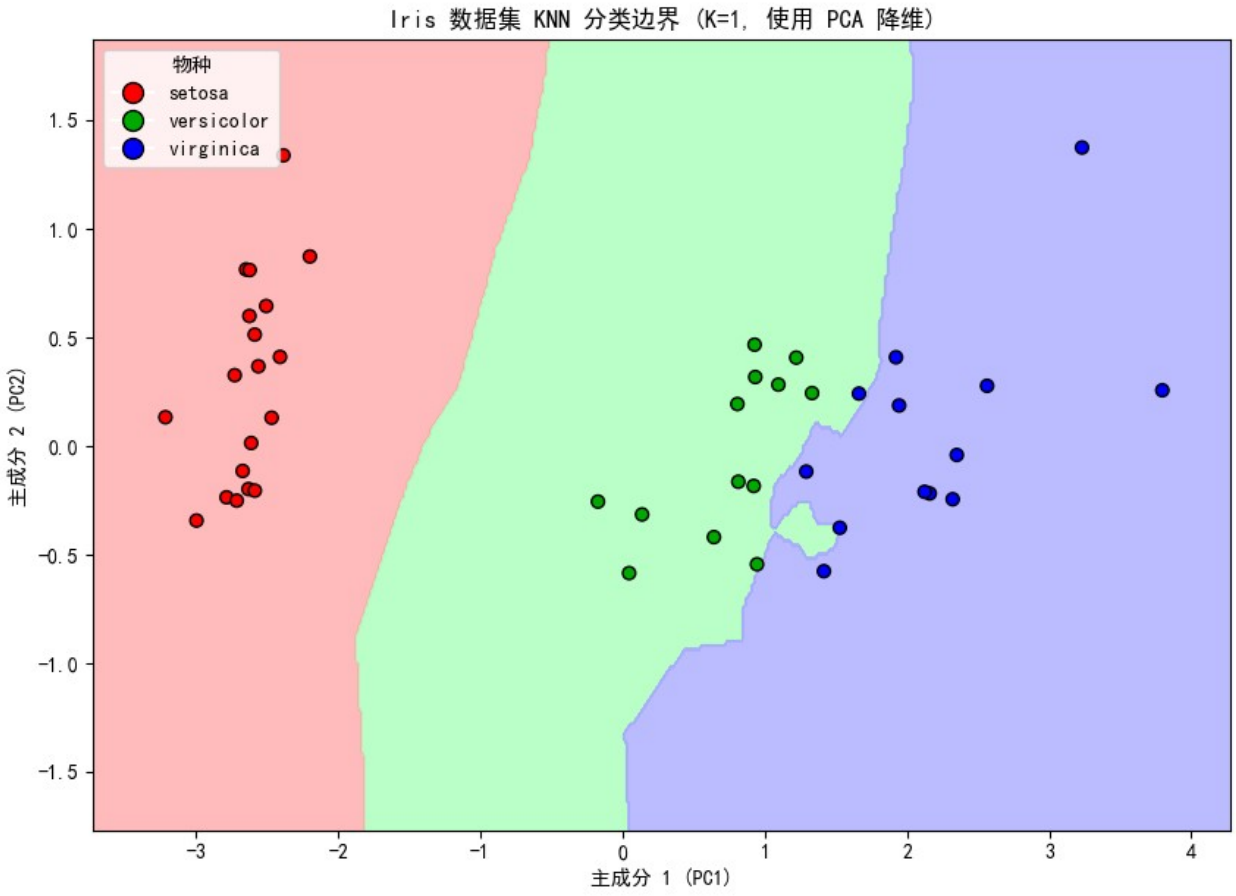
1. 使用 scikit-learn 加载经典 Iris（鸢尾花）数据集；
2. 应用 KNN（K-Nearest Neighbors）算法进行分类；
3. 调整 K 值并比较分类性能；
4. 对二维特征可视化分类边界

```
from matplotlib.lines import Line2D
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score
from matplotlib.colors import ListedColormap

# ============================================================
# 步骤1: 加载数据集、划分、PCA 降维
# ============================================================

# 加载Iris 数据集
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

# 划分训练集和测试集(70% 训练集, 30% 测试集)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 对全部数据进行PCA 降维，并分别划分降维后的训练集和测试集。
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

# 重新划分降维后的数据集
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(
    X_reduced, y, test_size=0.3, random_state=42
```

```
)


# ============================================================
# 步骤2 & 3: 应用KNN 算法、调整K 值并比较性能
# ============================================================

# 定义要测试的K 值列表
ks = [1, 5, 10, 15]
accuracies = {}

print("--- K 值调整和性能比较 ---")
for k in ks:
    # 使用原始4 维数据进行训练和测试(这是更准确的模型评估)
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    accuracies[k] = accuracy
    print(f"K={k}: 测试集准确率= {accuracy:.4f}")

# 找到最佳K 值
best_k = max(accuracies, key=accuracies.get)
print(f"\n最佳K 值(基于测试集准确率): {best_k}")


# ============================================================
# 步骤4: 可视化分类边界(使用降维后的数据)
# ============================================================

# 重新训练最佳K 值的模型,但使用2D PCA 数据进行可视化
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train_pca, y_train_pca)

# 定义颜色映射
cmap_light = ListedColormap(['#FFAAAA', '#AAFFBB', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00AA00', '#0000FF'])

# 1. 创建网格
h = 0.02
x_min, x_max = X_reduced[:, 0].min() - 0.5, X_reduced[:, 0].max() +
0.5
y_min, y_max = X_reduced[:, 1].min() - 0.5, X_reduced[:, 1].max() +
0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))

# 2. 对网格点进行预测
```

```
grid_data = np.c_[xx.ravel(), yy.ravel()]
Z = knn_best.predict(grid_data)
Z_mesh = Z.reshape(xx.shape)

# 3. 绘图
fig, ax = plt.subplots(figsize=(10, 7))

# 绘制决策区域(使用plt.contourf)
ax.contourf(xx, yy, Z_mesh, cmap=cmap_light, alpha=0.8)

# 叠加原始数据点(使用测试集数据)
scatter = ax.scatter(X_test_pca[:, 0], X_test_pca[:, 1], c=y_test_pca,
cmap=cmap_bold, edgecolor='k', s=40)

ax.set_title(f'Iris 数据集KNN 分类边界(K={best_k}, 使用PCA 降维)')
ax.set_xlabel('主成分1 (PC1)')
ax.set_ylabel('主成分2 (PC2)')
color_0 = cmap_bold(0)
color_1 = cmap_bold(1)
color_2 = cmap_bold(2)

legend_handles = [
    Line2D([0], [0], marker='o', color='w', label=target_names[0],
           markerfacecolor=color_0, markersize=10,
markeredgecolor='k'),
    Line2D([0], [0], marker='o', color='w', label=target_names[1],
           markerfacecolor=color_1, markersize=10,
markeredgecolor='k'),
    Line2D([0], [0], marker='o', color='w', label=target_names[2],
           markerfacecolor=color_2, markersize=10,
markeredgecolor='k')
]

ax.legend(handles=legend_handles, title="物种")

plt.show()
```

```
--- K 值调整和性能比较 ---
K=1: 测试集准确率= 1.0000
K=5: 测试集准确率= 1.0000
K=10: 测试集准确率= 1.0000
K=15: 测试集准确率= 1.0000

最佳K 值(基于测试集准确率): 1
```


Iris 数据集 KNN 分类边界（K=1，使用 PCA 降维）
物种
setosa
versicolor
virginica
主成分 2（PC2）
主成分 1（PC1）
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−3
−2
−1
0
1
2
3
4